

Introduction

Welcome to **YShared**! This is a basic example page demonstrating the various Markdown features supported by the documentation.

Headings

Markdown supports multiple heading levels:

Heading 1

Heading 2

Heading 3

Heading 4

Heading 5

Heading 6

Text Formatting

You can use **bold text**, *italic text*, and ***bold italic text***.

You can also use ~~strikethrough text~~.

You can highlight `inline code` using backticks.

You can combine formatting, such as **bold with inline code**.

Links

You can create links to other pages:

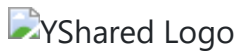
[Getting Started](#)

You can also link to external websites:

[Unity](#)↗

Images

Images can be embedded using:



You can also add alternative text describing the image.

Lists

Unordered List

- Item one
- Item two
- Item three
 - Nested item
 - Another nested item
 - Even deeper

Ordered List

1. First item
2. Second item
3. Third item
 1. Nested item
 2. Another nested item

Task List

- [x] Completed task
 - [] Incomplete task
 - [] Another task
-

Blockquotes

This is a blockquote.

It can span multiple lines.

Blockquotes can also be nested.

Code

Inline code:

```
Debug.Log("Hello World");
```

C# Code Block

```
using UnityEngine;

public class Example : MonoBehaviour
{
    private void Start()
    {
        Debug.Log("Hello World!");
    }
}
```

Although....

Getting Started

Quick Start

Welcome to **YShared**, a feature complete Dev Console system for Unity.

To get started, add the attribute [YCommand](#) to any function.

```
public class MyConsoleCommands : MonoBehaviour
{
    [YCommand]
    public int Add(int a, int b)
    {
        return a + b;
    }
}
```

and that's it! The command `add a:int b:int` is now added to the console.

YCommand Arguments

If you want the name of the console command to be different than the name of the function, you may change the name. You can also add a command description, which will be shown with the `help` command, when disambiguating between commands with the same name but different signature, when listing the usage of multiple subcommands, and when the user is informed there is an error in their input.

```
public class MyConsoleCommands : MonoBehaviour
{
    [YCommand("multiply")]
    public int SuperComplicatedMathematicalOperation(int a, int b)
    {
        return a * b;
    }
}
```

If the function is a static method, the command will simply run without an instance.

If the function is an instance method, an [ObjectFindType](#) can be given to tell YConsole how to find the object to run this command on. By default, `ObjectFindType.All` is used.

Value	Description
<code>All</code>	Finds all objects of this class and calls the method on each of them. Internally uses <code>GameObject.FindObjectsByType</code> .

Value	Description
Any	Finds any object of this class and calls the method on it. Internally uses <code>GameObject.FindAnyByType</code> .
First	Finds the first object of this class and calls the method on it. Internally uses <code>GameObject.FindFirstByType</code> .
Argument	Uses the first argument of the command as the instance to run the method on. The first argument does not need to be explicitly typed as the class; if it is, YConsole automatically fills it in.
ArgumentArray	Uses the first argument of the command as an array of instances to run the method on. The first argument does not need to be explicitly typed as an array of the class; if it is, YConsole automatically fills it in.